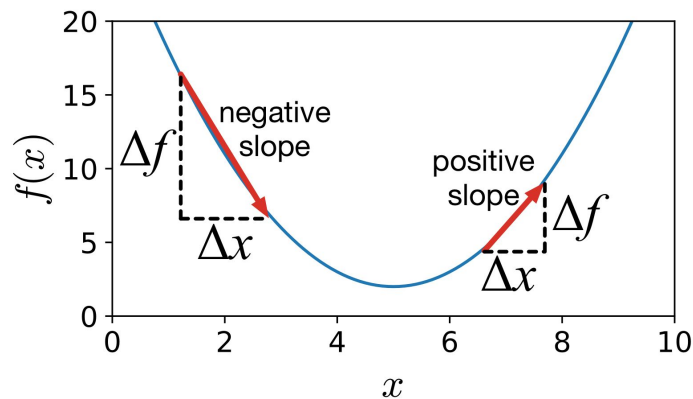
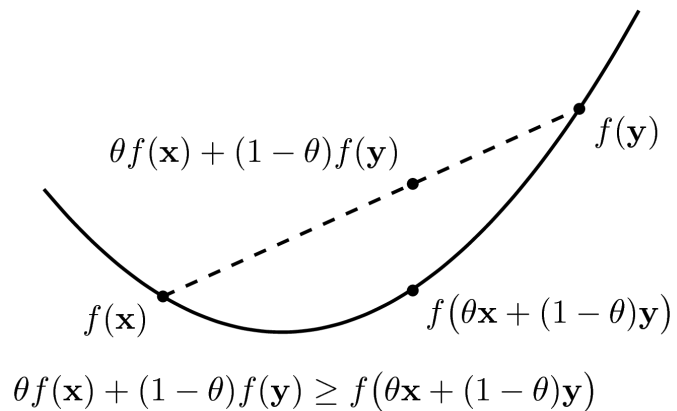
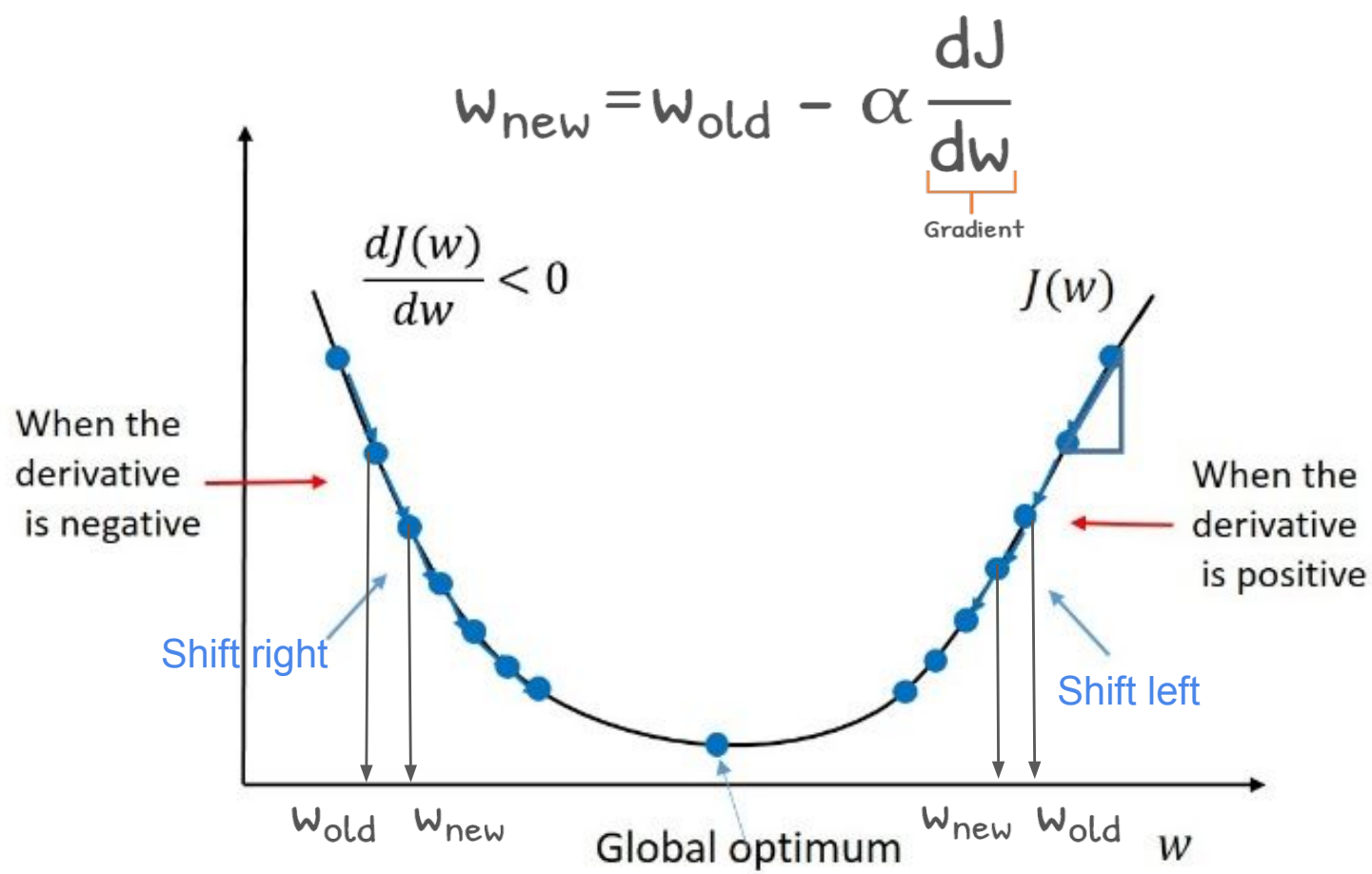


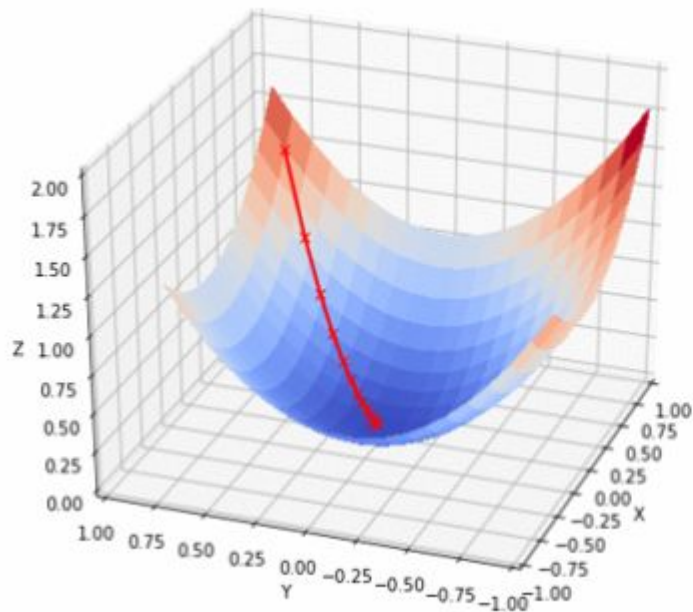
Deep Learning

lec-2

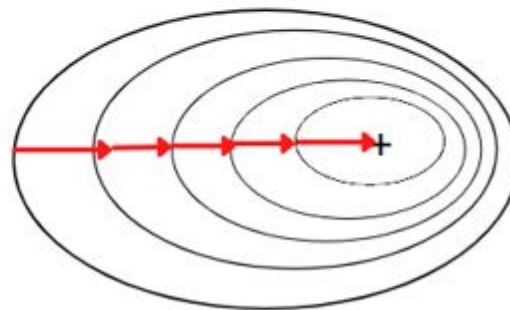
Gradient descent



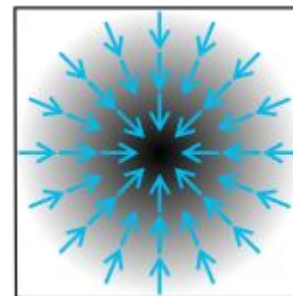




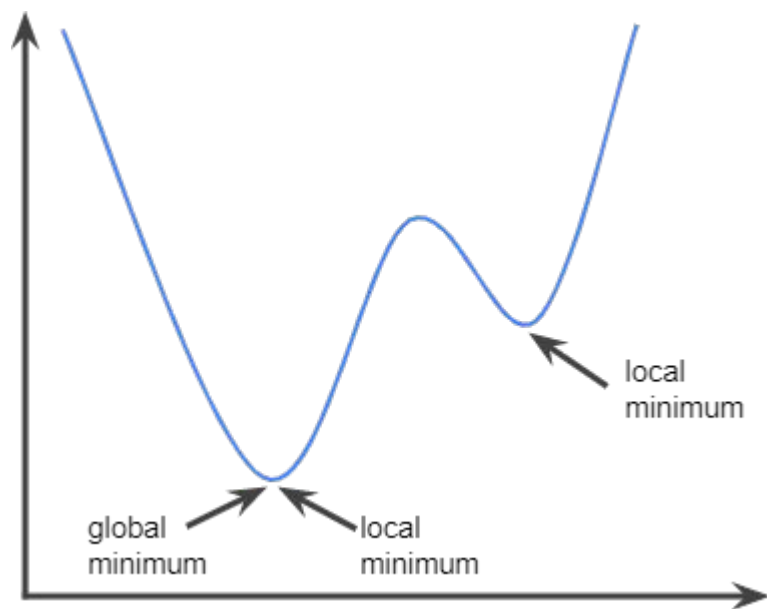
Contours

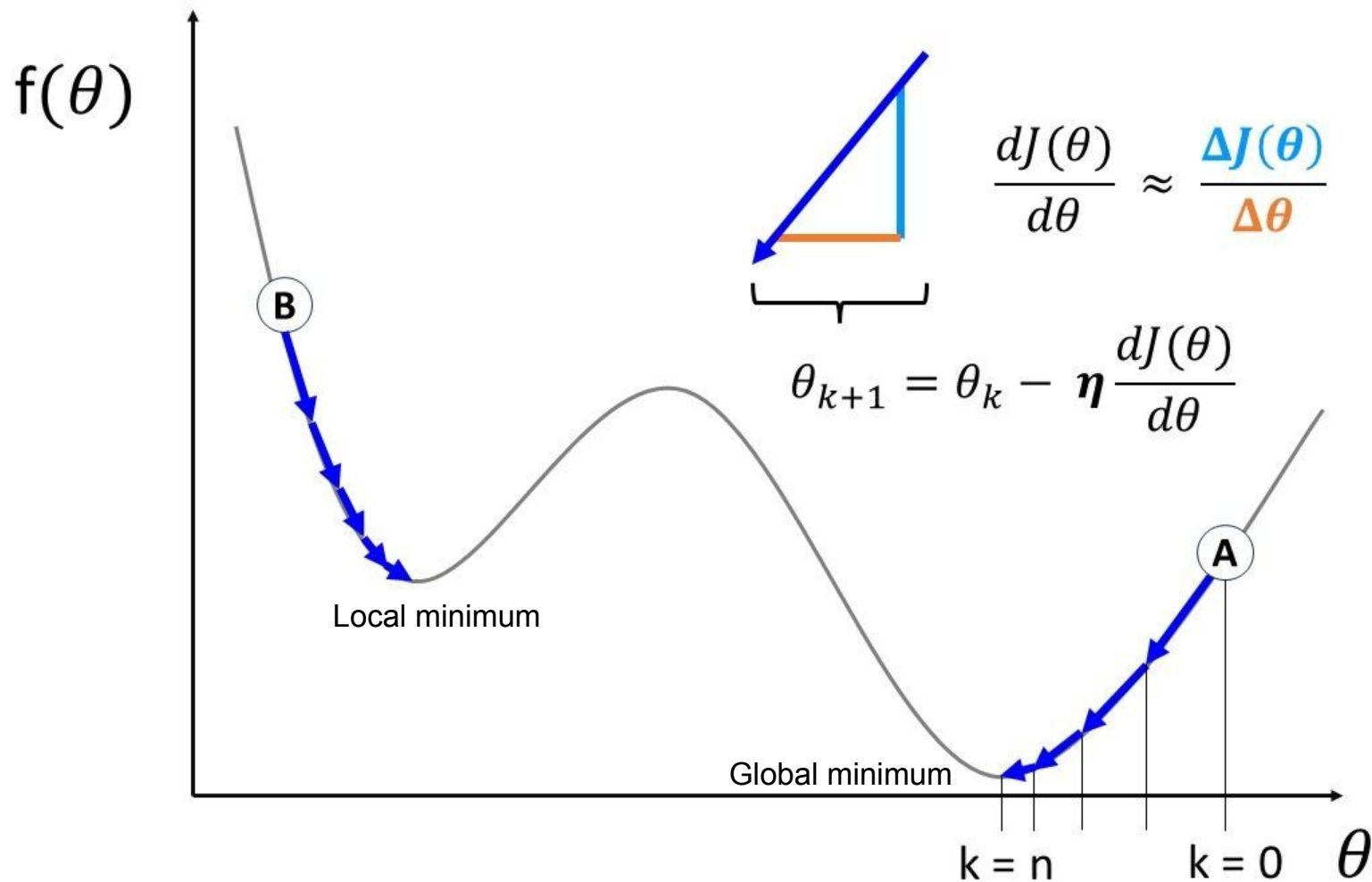


Gradient flow for a perfect convex surface

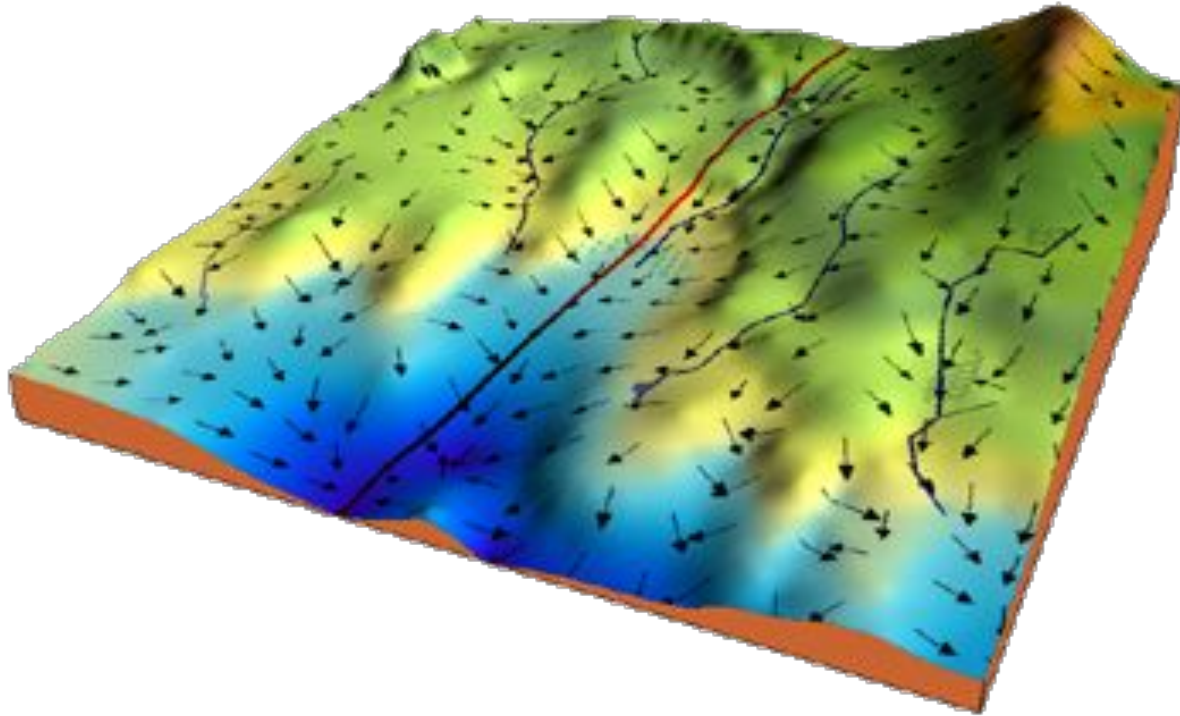


Non convex functions





In general objective is to achieve Steepest descent point overcoming local minima:
Optimizers



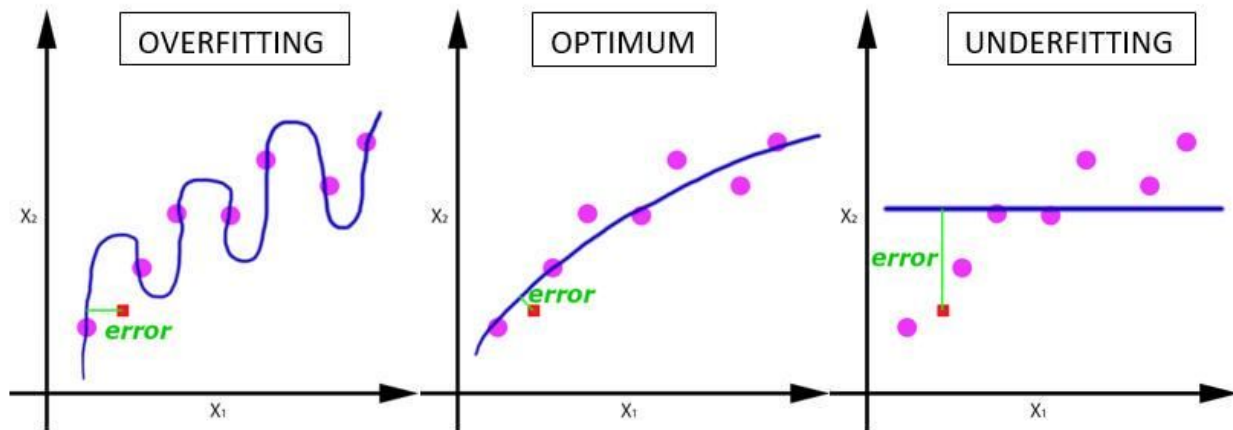
Feed Forward Neural Network:

<https://drive.google.com/file/d/1OJhGgrjIG30IzpCLvuwx8HuJ5hP3dkC5/view?usp=sharing>

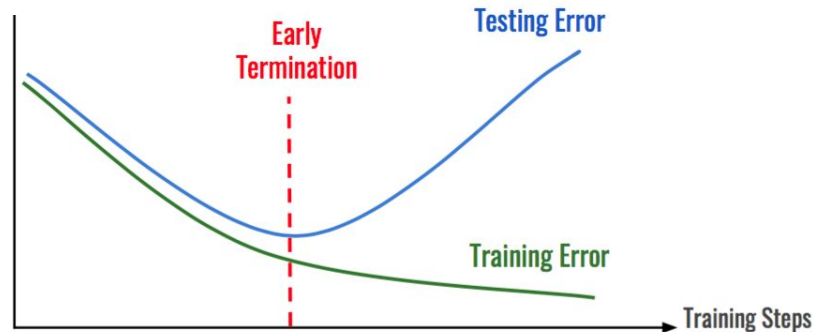
Item	Feed-Forward NN (BP)
Forward pass	$h = f(W^{(1)}x)$ $y = g(W^{(2)}h)$
Loss	$L = \ell(y, \hat{y})$
Output-layer error	$\delta^{(2)} = \frac{\partial L}{\partial a^{(2)}}$
Hidden-layer error	$\delta^{(1)} = (W^{(2)})^\top \delta^{(2)} \odot f'(a^{(1)})$
Weight update – output	$\Delta W^{(2)} = -\eta \delta^{(2)} h^\top$
Weight update – input/hidden	$\Delta W^{(1)} = -\eta \delta^{(1)} x^\top$

Training aspects of Neural Networks

Overfitting and regularization

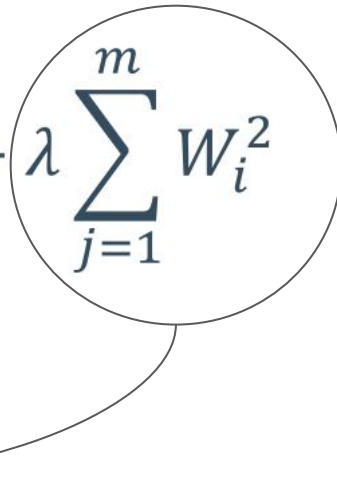


- Model keeps increasing weight magnitudes to drive training error toward zero.
- Memorization via Large Weights.
- Poor generalization.

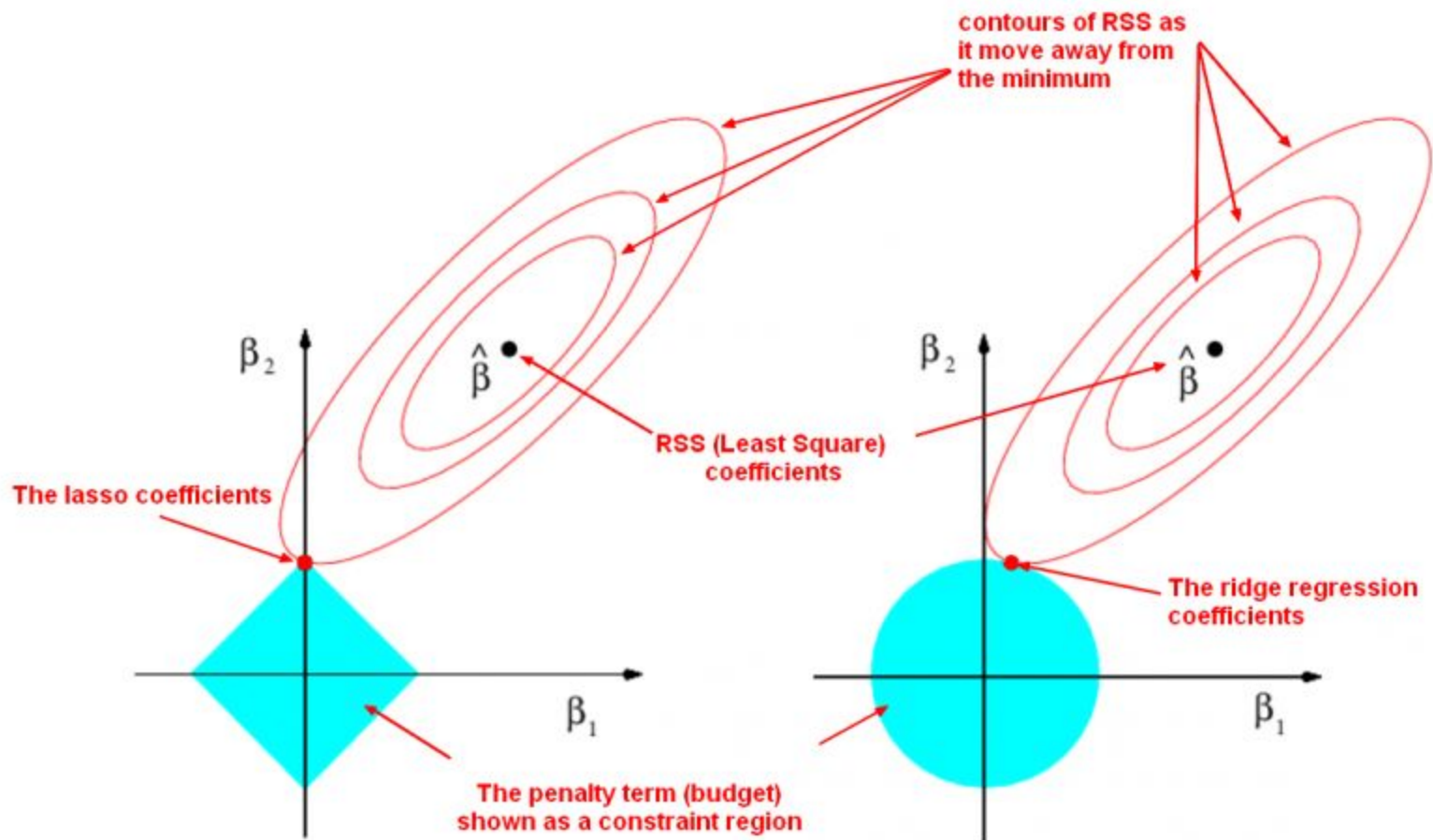


Regularization (penalty on large weights)

Regularization penalty in cost function

$$J = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^m w_j^2$$


- Lasso or L1-regression $|w|$
- Ridge or L2 regression

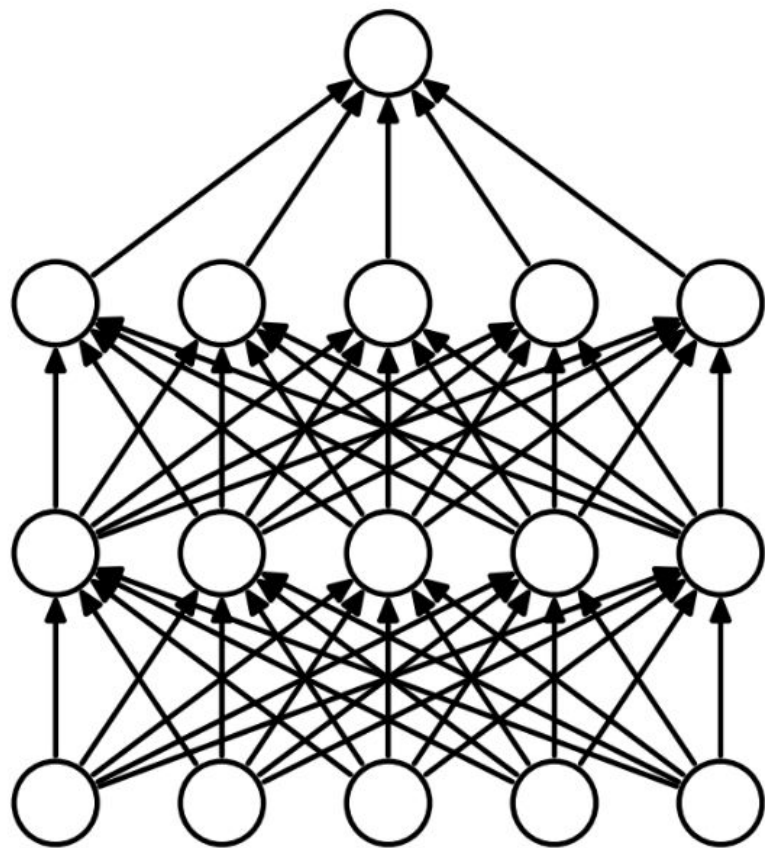


LASSO

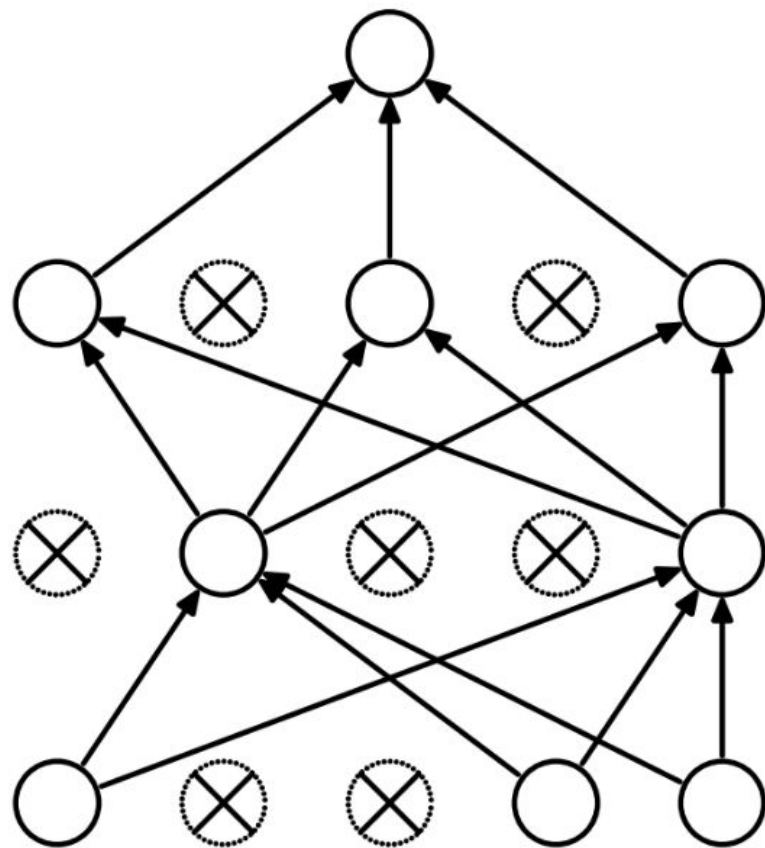
RIDGE REGRESSION

Dropout

- Dropout is a mechanism where at each training iteration (batch) we randomly remove a subset of neurons
- This prevents the neural network from relying too much on individual pathways, making it more “robust”
- At test time we “rescale” the weight of the neuron to reflect the percentage of the time it was active

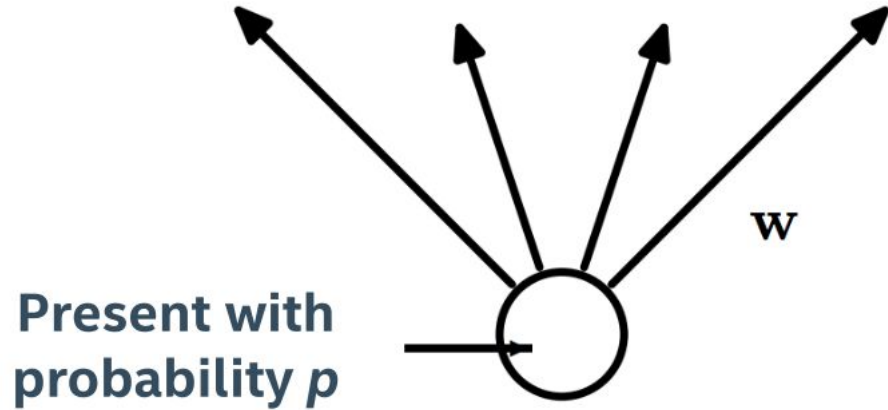


(a) Standard Neural Net

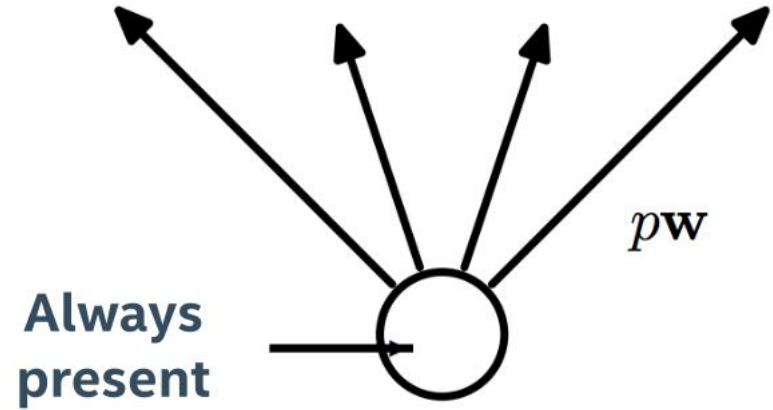


(b) After applying dropout

If the neuron was present with probability p , at test time we scale the outbound weights by a factor of p .



(a) At training time



(b) At test time

Early stopping

Another, more heuristic approach to regularization is early stopping.

This refers to choosing some rules after which to stop training.

Example:

- Check the validation log-loss every 10 epochs.
- If it is higher than it was last time, stop and use the previous model (i.e. from 10 epochs previous)

Optimizers

- We have considered approaches to gradient descent which vary the number of data points involved in a step.
- However, they have all used the standard update formula:

$$W \leftarrow W - \alpha \cdot \nabla J$$

- There are several variants to updating the weights which give better performance in practice.
- These successive “tweaks” each attempt to improve on the previous idea.
- The resulting (often complicated) methods are referred to as “optimizers”.

Momentum

Idea, only change direction by a little bit each time.

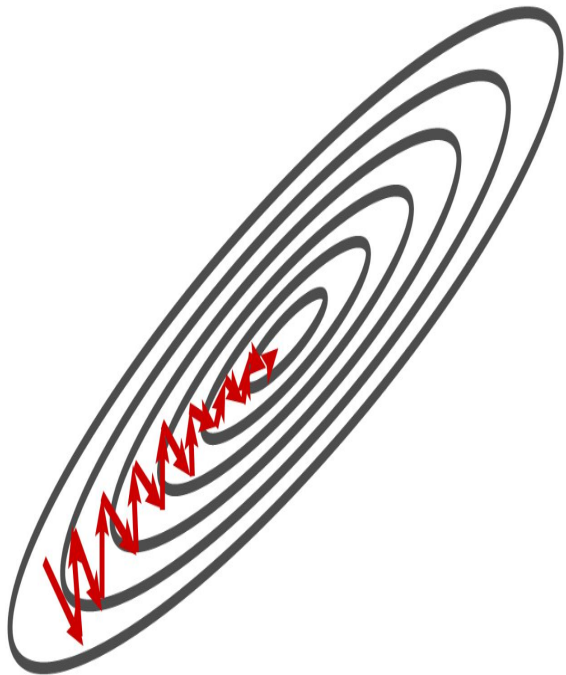
Keeps a “**running average**” of the step directions, smoothing out the variation of the individual points.

$$\mathcal{V}_t \coloneqq \eta \cdot \mathcal{V}_{t-1} - \alpha \cdot \nabla J$$

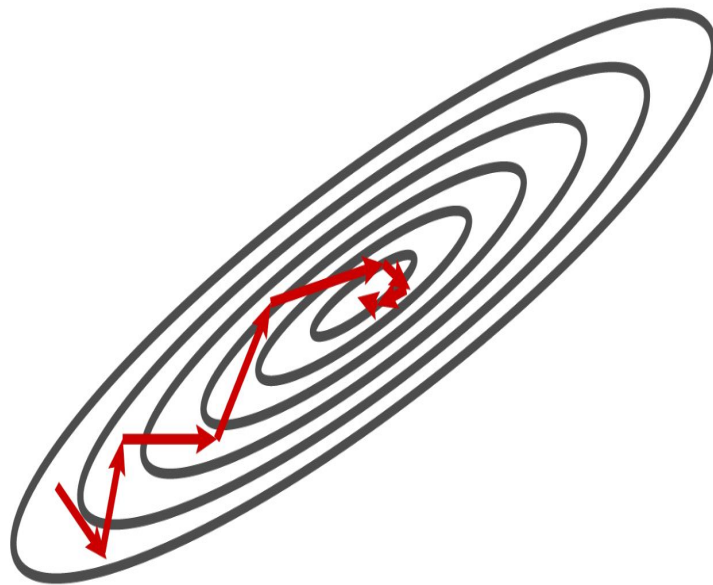
$$W \coloneqq W - \mathcal{V}_t$$

Here, η is referred to as the “momentum”. It is generally given a value <1 [0.9,0.99]

- Consistent gradients $\rightarrow$ speed up
- Oscillating gradients $\rightarrow$ cancel out



Gradient descent



Momentum

Adam

Idea: use both first-order and second-order change information and decay both over time. Step size depends on gradient variance (1st and 2nd order).

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla J & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) \nabla J^2 \\ &\downarrow & &\downarrow \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} & \hat{v}_t &= \frac{v_t}{1 - \beta_1^t} \\ &\searrow & &\searrow \\ W &:= W - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \odot \hat{m}_t \end{aligned}$$

Data Augmentation

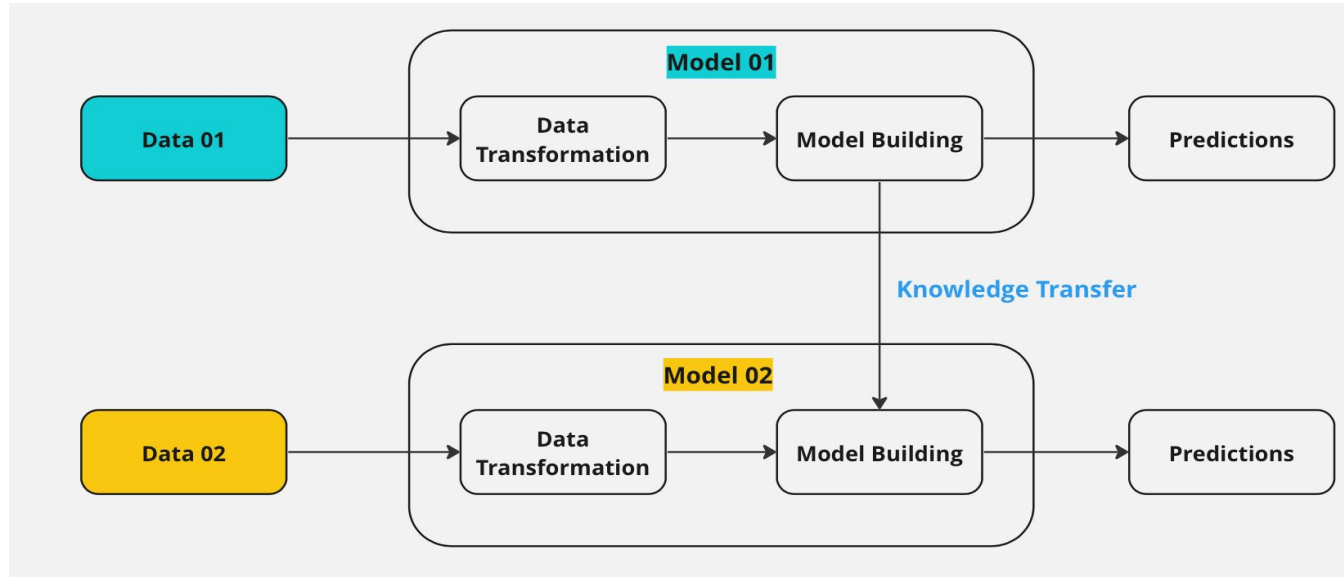
Increase data by

- Rotate
- Flip
- Shift
- Noise



Transfer learning

- Model trained on one task is reused as the foundation for a second task.
- This approach is beneficial when
 - The second task is related to the first (source/task1 and target/task2 data distributions are different)
 - When data for the second task is limited.

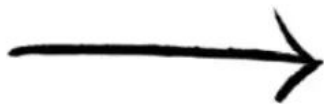


Pre-Training

(Computationally Expensive)



Large
Unlabeled Corpus



Fine-Tuning

(Cheaper)



Small
Labeled Corpus

Steps to perform Transfer Learning

- Start with a model already trained on a large dataset for a specific task call it as **Pre-trained model**.
- Identify the layers within the pre-trained model that hold generic information: **Frozen layers**.
- **Append** the task specific layers to these generic layers
- **Fine-tune**
 - these selected layers with data from the new task.
 - Sometimes whole network can be fine tuned
- This process helps retain the pre-trained knowledge while adjusting parameters to meet the specific requirements of the new task

